

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Mateus dos Santos Pereira

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	2
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	6
4.4	Atividade 4 — Controle de acesso por papel — RBAC	10
4.5	Atividade 5 — Logs e auditoria	13
4.6	Atividade 6 — Upload e perfil de usuário	15
5	Considerações finais	17
6	Declaração de autoria	18

1 Introdução

Na disciplina ISW055 — **Introdução à Computação em Nuvem**, da Fatec Pompeia, a gente estuda o básico de como funcionam os sistemas na nuvem: microsserviços, contêineres, banco de dados, autenticação, controle de acesso e logs. O que guiou o bimestre inteiro foi um projeto só: um catálogo de filmes do Tom Hanks, que foi crescendo aos poucos. Começou simples e terminou virando um sistema com vários serviços rodando em contêineres, publicado na internet.

Foram seis atividades no total. Primeiro fizemos uma agenda bem simples em Flask, em sala, como nivelamento. Depois veio o catálogo de filmes consumindo a API do TMDb e salvando no MariaDB. Na sequência o login foi separado num microsserviço próprio, veio o controle de acesso por perfil (RBAC), os logs e a auditoria, e por último o perfil do usuário com foto salva no MinIO. Tudo ficou no mesmo repositório público do GitHub, com deploy num ambiente de produção.

Este relatório junta tudo isso num documento só: mostra o que era pra ser feito e quando, o que eu realmente entreguei e quando, com uma ficha por atividade trazendo o link da evidência e os prints comprovando. No final escrevo o que aprendi, o que foi mais difícil e o que eu faria diferente.

2 Metodologia

Todas as atividades foram feitas no mesmo repositório público do GitHub (mateussantospereira/filmes-tom-hanks), como continuação do mesmo projeto. Para programar usei Bun e o framework Hono nas APIs, o banco era MariaDB, os serviços rodavam em Docker Compose e o deploy era feito pelo Portainer. Os arquivos de upload foram para o MinIO e os logs de auditoria para o Redis, lidos por um serviço de logs separado. Cada atividade eu testava na minha máquina e, quando possível, publicava no ambiente de produção usado nas demonstrações (<https://mateus-pereira-isw055.lapps.studio>).

As datas e horas de cada entrega foram tiradas do histórico de commits do repositório com o comando `git log` (no formato DD/MM/AAAA HH:MM) e conferidas na página de cada commit no GitHub. Para cada atividade o relatório traz duas provas: um print do terminal com o `git log` (mostrando o hash, a data/hora e o repositório) e o print da própria página do commit no GitHub — além do print do sistema funcionando. O README do repositório documenta cada atividade, mostra a arquitetura e menciona o professor (github.com/siriani).

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência leve no navegador (localStorage).	07/08/2026	07/08/2026 (em sala)	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	18/08/2026 17:48	entregue
3	Desacoplando o login —	Login, cadastro e esqueci-minha-senha num serviço à	28/08/2026	25/09/2026 16:21	entregue com atraso

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
	microserviço de autenticação	parte na rede interna do Docker.			
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	06/10/2026 11:18	entregue com atraso
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	06/10/2026 18:30	entregue com atraso
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	06/10/2026 19:50	entregue com atraso

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência leve no navegador (localStorage).
Data planejada	07/08/2026
Data realizada	07/08/2026 (em sala)
Situação	entregue
Evidência	Realizada em sala — print do sistema rodando localmente
Link	—

O que foi feito. A primeira atividade foi um nivelamento em sala: uma agenda telefônica bem simples em Flask com templates Jinja, onde dá para cadastrar, listar, editar e remover contatos. Não tinha banco de dados — os contatos ficavam no `localStorage` do navegador, de propósito. Era só para a turma se acostumar com o padrão rota → função → template antes de começar o projeto grande.



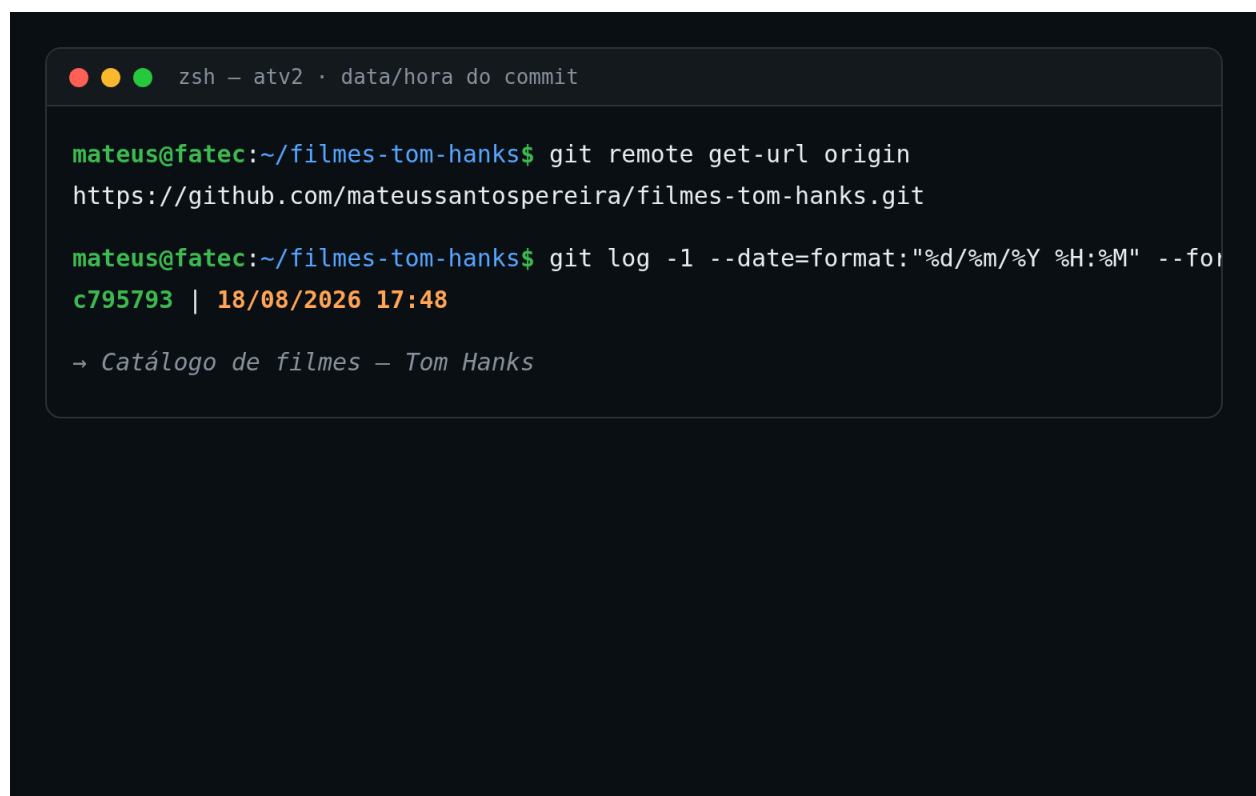
Figura 1: Atividade 1 — resultado: agenda telefônica em Flask rodando

Dificuldades e como foram resolvidas. Não teve dificuldade técnica, era um nivelamento. O código não foi para o GitHub na época, só rodou em sala, e o print de comprovação é o que está acima.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	18/08/2026 17:48
Situação	entregue
Evidência	GitHub — commit + README + print do catálogo
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/c795793

O que foi feito. Criei o repositório e o catálogo de filmes do Tom Hanks: a aplicação busca os filmes na API do TMDb, guarda os dados no MariaDB e tem uma área logada onde cada usuário tem os próprios favoritos (segregação por usuário). Montei também o docker-compose para a aplicação e o banco rodarem juntos, com deploy via Portainer. O README documenta a arquitetura, o uso e o endereço de produção, e menciona o professor.



```
zsh - atv2 · data/hora do commit

mateus@fatec:~/filmes-tom-hanks$ git remote get-url origin
https://github.com/mateussantospereira/filmes-tom-hanks.git

mateus@fatec:~/filmes-tom-hanks$ git log -1 --date=format:"%d/%m/%Y %H:%M" --for
c795793 | 18/08/2026 17:48

→ Catálogo de filmes — Tom Hanks
```

Figura 2: Atividade 2 — evidência: commit c795793 (terminal, 18/08/2026 17:48)

The screenshot shows a GitHub commit page for the repository 'filmes-tom-hanks' by user 'mateussantospereira'. The commit hash is 'c795793'. The commit message is 'Tom Hanks'. The commit is linked to 0 parents. The commit shows 19 files changed with a total of +1,087 lines added. The file list on the left includes: .env.example, .gitignore, Dockerfile, README.md, drizzle.config.ts, package.json, src, db, index.ts, schema.ts, middleware, auth.ts, and public. The diff view shows changes to .env.example (8 lines added) and .gitignore (3 lines added). The .env.example file contains: TMDB_API_KEY=, DB_HOST=, DB_PORT=3306, DB_USER=, DB_PASSWORD=, DB_NAME=, JWT_SECRET=, and PORT=3000. The .gitignore file contains: node_modules/, .env, and *.lock.

Platform Solutions Resources Open Source Enterprise Pricing

mateussantospereira / filmes-tom-hanks

Code Issues 4 Pull requests Actions Projects Security and quality Insights

Commit c795793

mateussantospereira committed

Tom Hanks

0 parents commit c795793

19 files changed +1,087

Filter files...

.env.example +8

```
... @@ -0,0 +1,8 @@
1 + TMDB_API_KEY=
2 + DB_HOST=
3 + DB_PORT=3306
4 + DB_USER=
5 + DB_PASSWORD=
6 + DB_NAME=
7 + JWT_SECRET=
8 + PORT=3000
```

.gitignore +3

```
... @@ -0,0 +1,3 @@
1 + node_modules/
2 + .env
3 + *.lock
```

Dockerfile +14

Figura 3: Atividade 2 — evidência: página do commit no GitHub

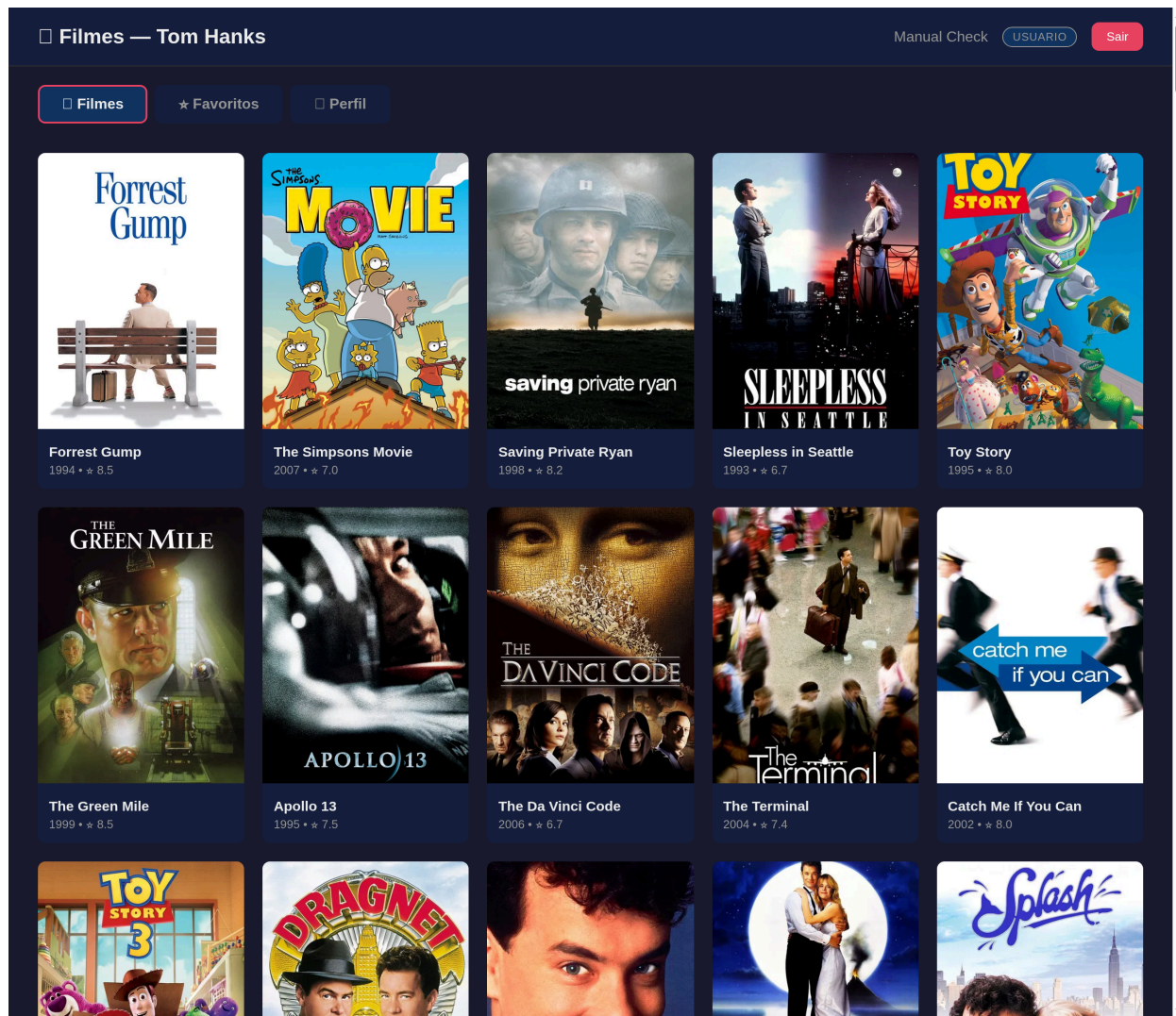


Figura 4: Atividade 2 — resultado: catálogo logado em produção

Dificuldades e como foram resolvidas. A primeira montagem do ambiente deu trabalho: dependências e a rede do Docker. Resolvi separando o lockfile e o compose em commits próprios, para o build ficar determinístico e repetível no Portainer.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	25/09/2026 16:21
Situação	entregue com atraso
Evidência	GitHub — commit + docker-compose.yml + print do login funcionando
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/d08ced9

O que foi feito. Extraí a autenticação do catálogo para um serviço separado (auth-service), que passou a ser o único dono da tabela de usuários e da lógica de senha, JWT e papel. O catálogo virou um gateway: as rotas de login, cadastro, esqueci minha senha e reset só repassam a chamada para o auth-service

pela rede interna do Docker (AUTH_SERVICE_URL). A recuperação de senha envia e-mail de verdade pelo Mailtrap, e o docker-compose.yml documenta os dois serviços.

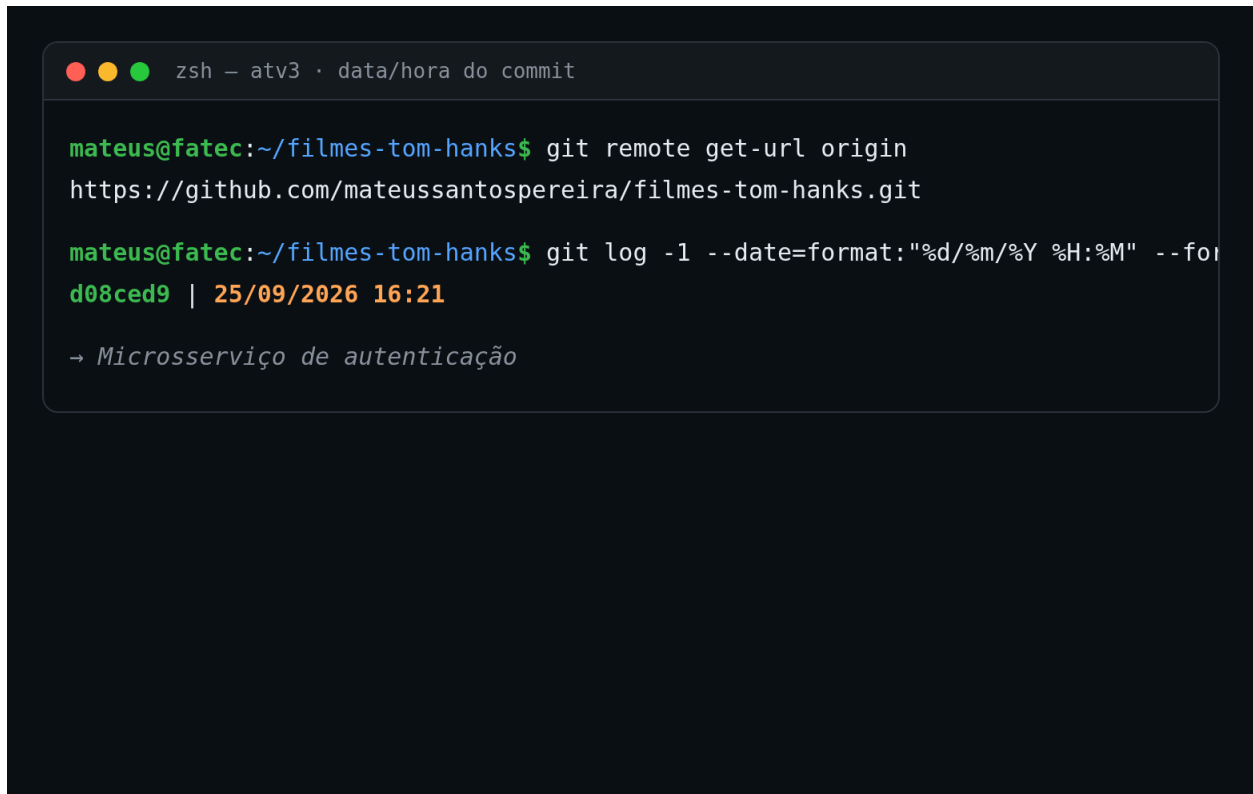
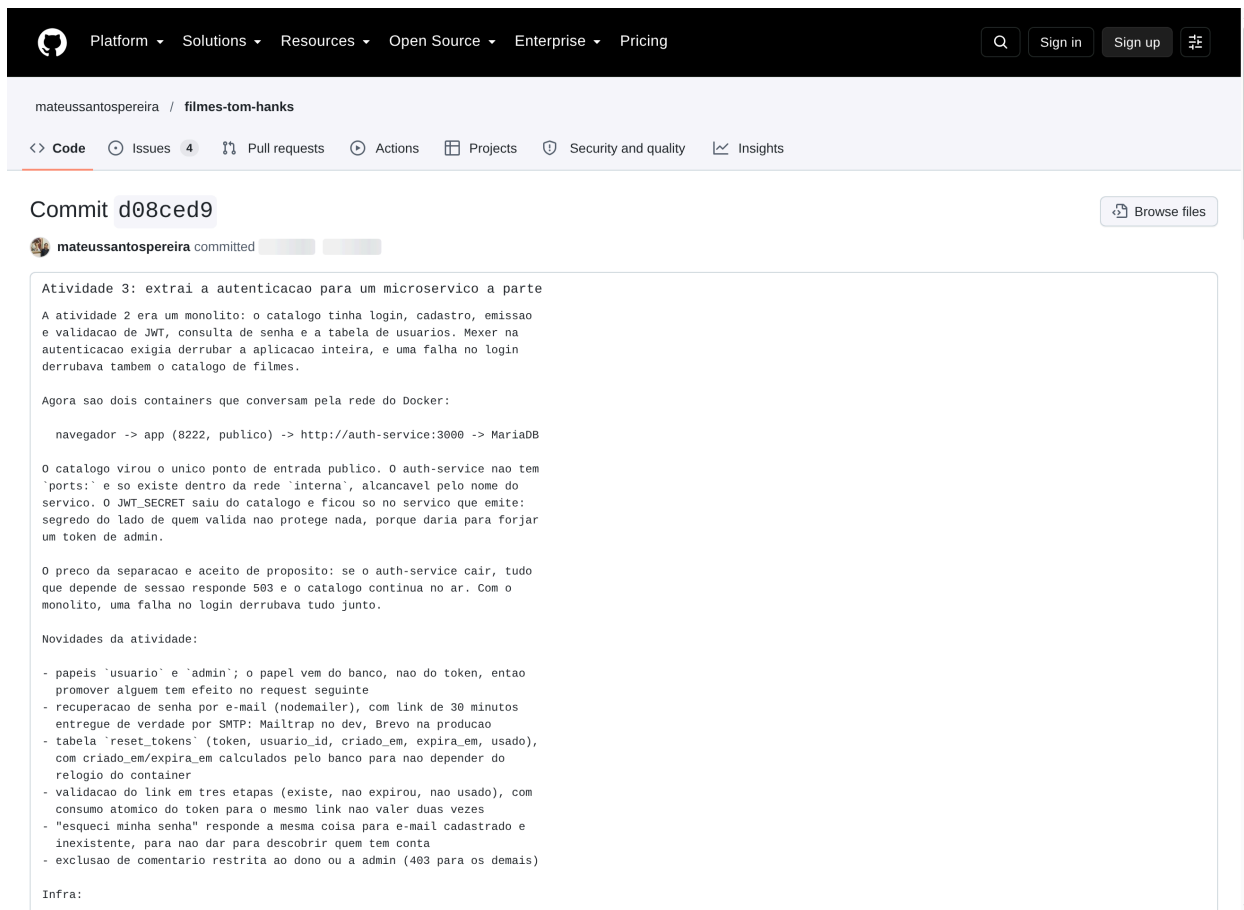
A terminal window with a dark background and light-colored text. The title bar at the top shows three colored circles (red, yellow, green) followed by the text 'zsh - atv3 · data/hora do commit'. The terminal content shows two commands being executed. The first command is 'git remote get-url origin', which returns 'https://github.com/mateussantospereira/filmes-tom-hanks.git'. The second command is 'git log -1 --date=format:"%d/%m/%Y %H:%M" --for', which returns 'd08ced9 | 25/09/2026 16:21'. Below the output, there is a line of text '→ Microserviço de autenticação'.

Figura 5: Atividade 3 — evidência: commit d08ced9 (terminal, 25/09/2026 16:21)



The screenshot shows a GitHub commit page for the repository 'mateussantospereira / filmes-tom-hanks'. The commit hash is 'd08ced9'. The commit message is 'Atividade 3: extrai a autenticao para um microservico a parte'. The commit was made by 'mateussantospereira'.

The commit message details the development of a microservice for authentication, separating it from a monolithic application. It describes the initial state where the catalog and authentication were intertwined, and the goal was to create a public entry point for the catalog and a separate authentication service.

The commit message includes the following text:

```
Atividade 3: extrai a autenticao para um microservico a parte

A atividade 2 era um monolito: o catalogo tinha login, cadastro, emissao
e validacao de JWT, consulta de senha e a tabela de usuarios. Mexer na
autenticacao exigia derrubar a aplicacao inteira, e uma falha no login
derrubava tambem o catalogo de filmes.

Agora sao dois containers que conversam pela rede do Docker:

navegador -> app (8222, publico) -> http://auth-service:3000 -> MariaDB

O catalogo virou o unico ponto de entrada publico. O auth-service nao tem
'ports:' e so existe dentro da rede 'interna', alcancavel pelo nome do
servico. O JWT_SECRET saiu do catalogo e ficou so no servico que emite:
segredo do lado de quem valida nao protege nada, porque daria para forjar
um token de admin.

O preco da separacao e aceito de proposito: se o auth-service cair, tudo
que depende de sessao responde 503 e o catalogo continua no ar. Com o
monolito, uma falha no login derrubava tudo junto.

Novidades da atividade:

- papeis 'usuario' e 'admin'; o papel vem do banco, nao do token, entao
promover alguem tem efeito no request seguinte
- recuperacao de senha por e-mail (nodemailer), com link de 30 minutos
entregue de verdade por SMTP: Mailtrap no dev, Brevo na producao
- tabela 'reset_tokens' (token, usuario_id, criado_em, expira_em, usado),
com criado_em/expira_em calculados pelo banco para nao depender do
relogio do container
- validacao do link em tres etapas (existe, nao expirou, nao usado), com
consumo atomico do token para o mesmo link nao valer duas vezes
- "esqueci minha senha" responde a mesma coisa para e-mail cadastrado e
inexistente, para nao dar para descobrir quem tem conta
- exclusao de comentario restrita ao dono ou a admin (403 para os demais)

Infra:
```

Figura 6: Atividade 3 — evidência: página do commit no GitHub

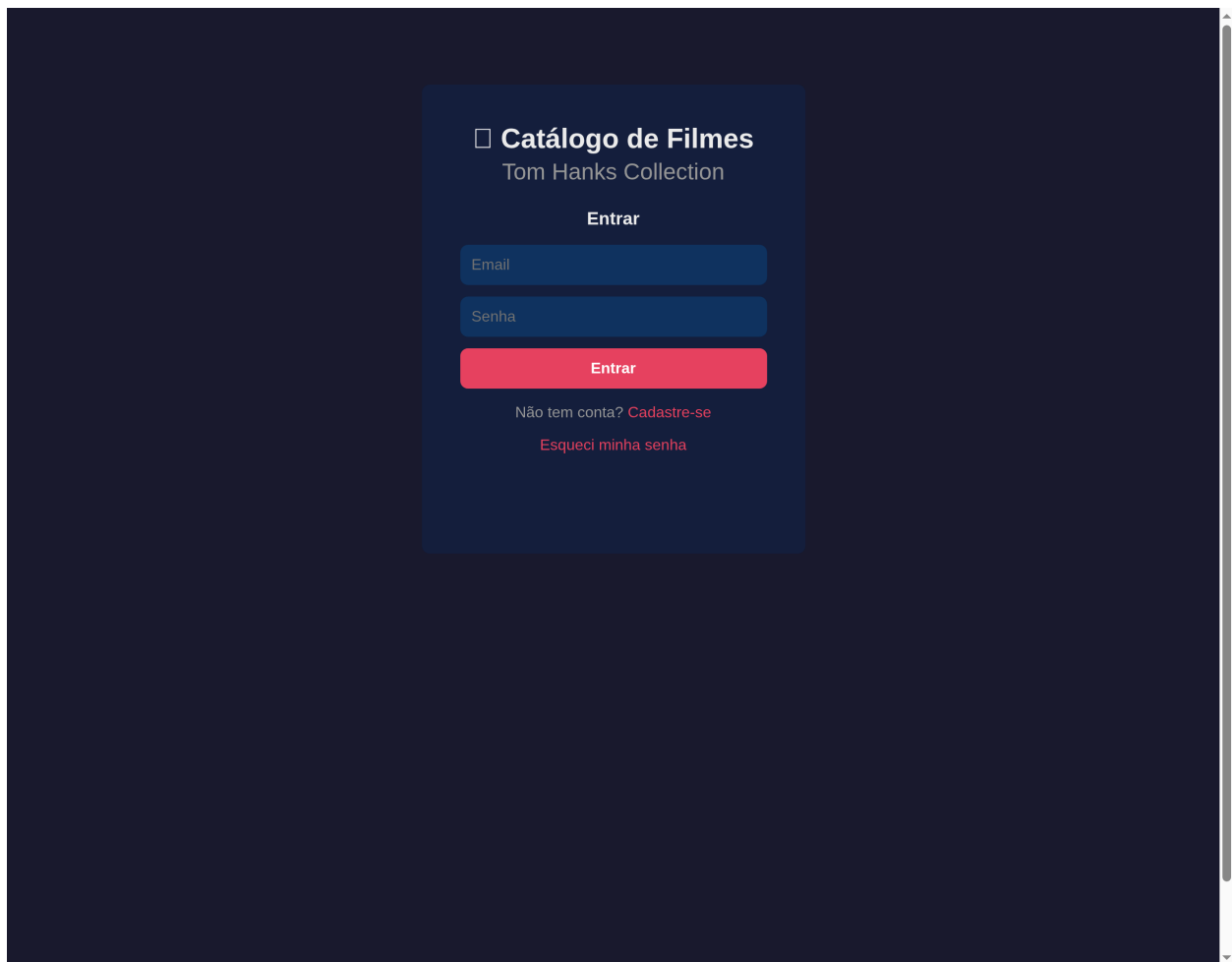


Figura 7: Atividade 3 — resultado: tela de login em produção

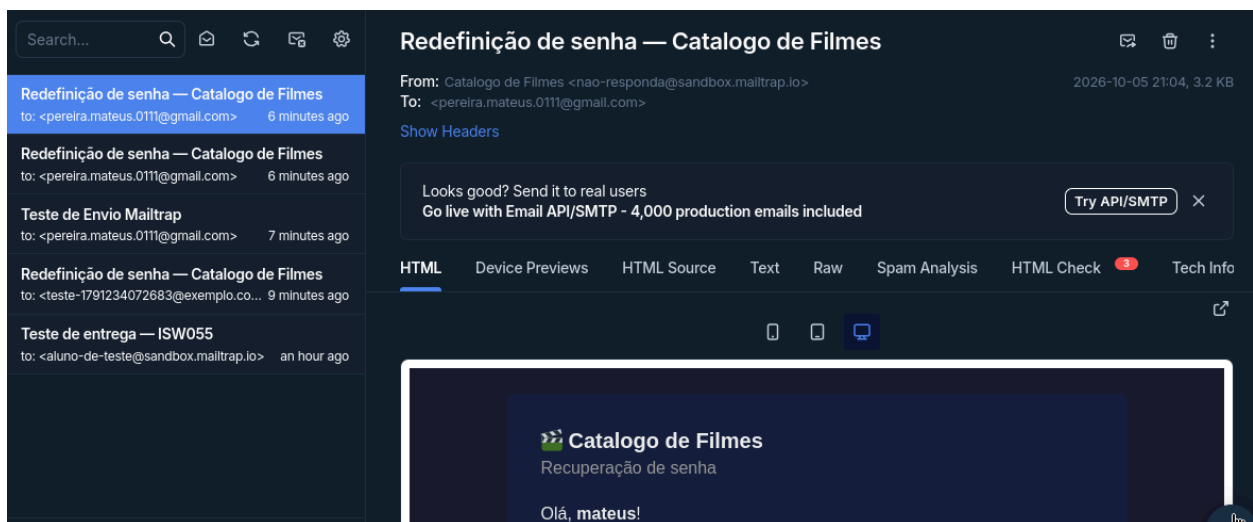


Figura 8: Atividade 3 — resultado: e-mail de recuperação de senha (Mailtrap)

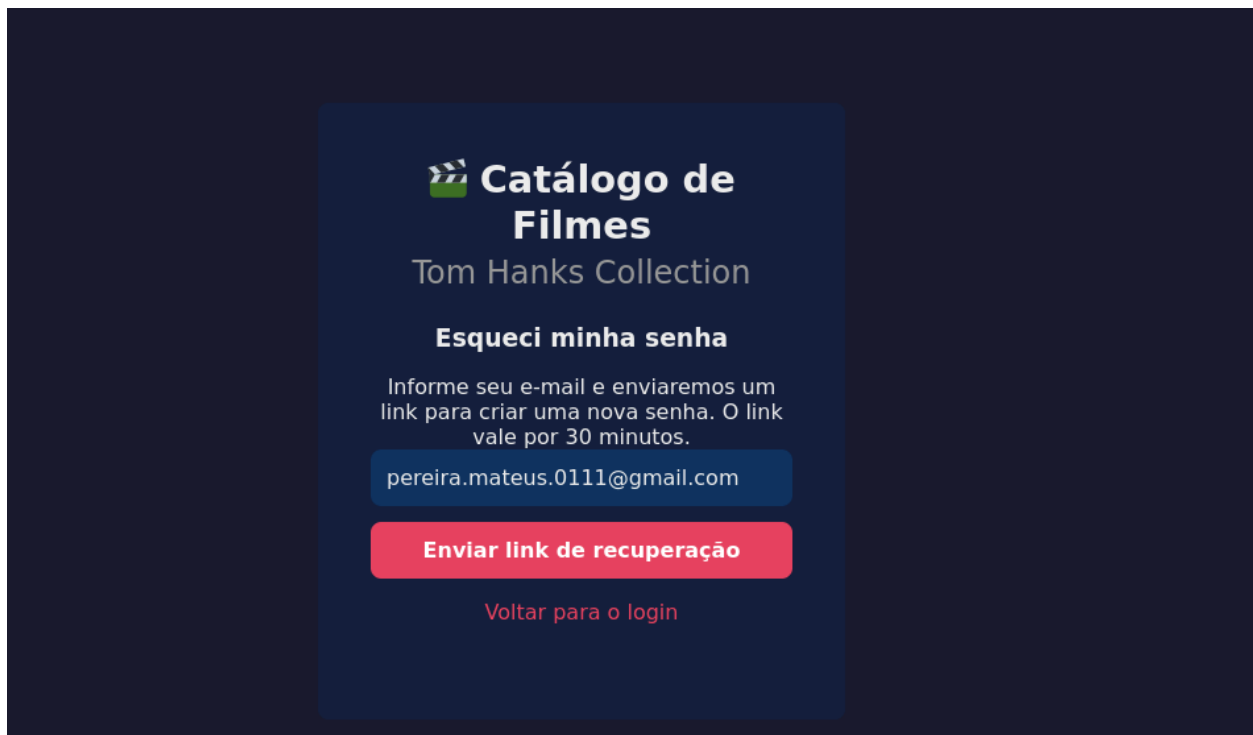


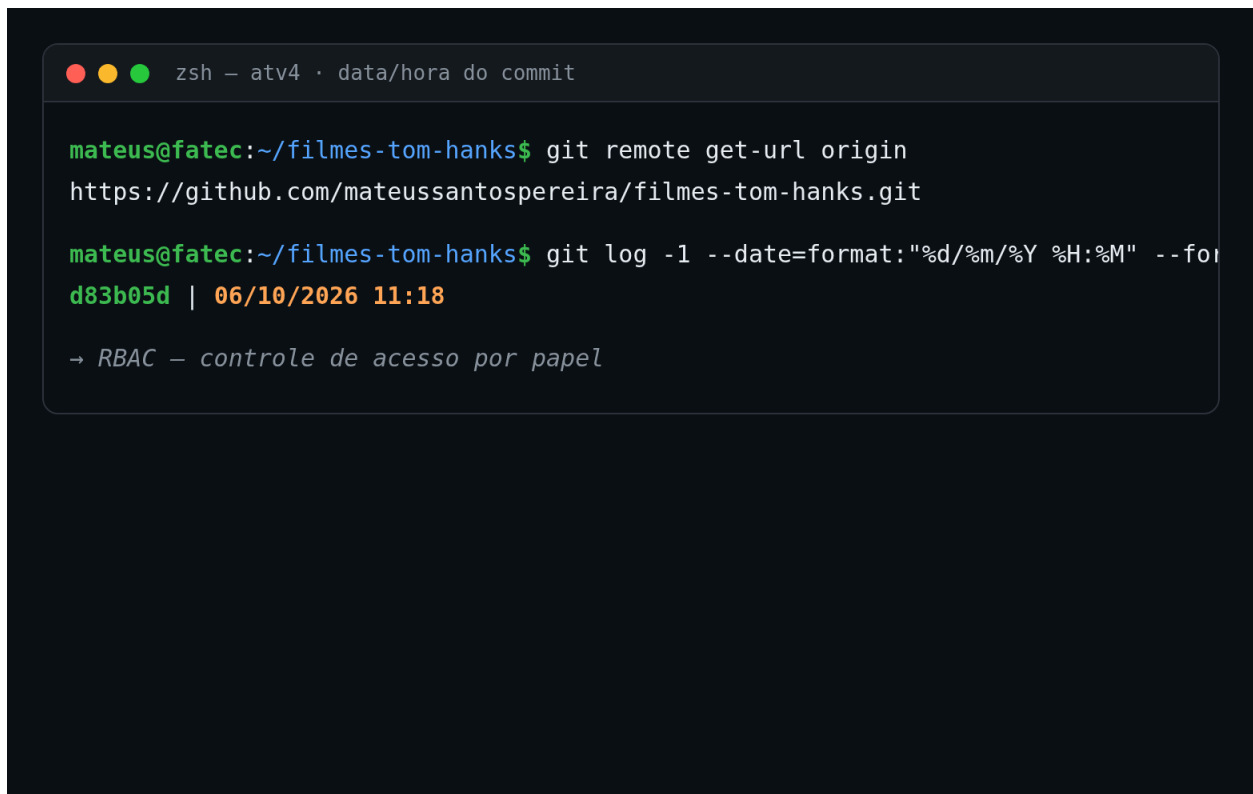
Figura 9: Atividade 3 — resultado: fluxo de redefinição de senha

Dificuldades e como foram resolvidas. Sincronizar dois serviços numa rede interna sem expor o auth-service para fora exigiu cuidado com a variável `AUTH_SERVICE_URL` (nome do serviço, não localhost) e com timeouts, para não travar a requisição do usuário. Os ajustes finos dos defaults e das dependências ficaram para a semana seguinte, o que atrasou a entrega.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	06/10/2026 11:18
Situação	entregue com atraso
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/d83b05d

O que foi feito. O campo role deixou de ser só enfeite e passou a valer de verdade no servidor: criei um mapa de permissões (`auth-service/src/auth/permissions.ts`), onde admin é superconjunto de usuario e qualquer permissão ausente nega (sempre **fail-closed**). O GET `/me` devolve as permissões resolvidas e o catálogo checa `permissions.includes(...)` em vez de comparar `role === "admin"`. Ações exclusivas de admin (apagar comentário de terceiro, listar usuários, mudar papel) devolvem 403 para usuário comum. Também consertei um XSS guardado em comentários, trocando `innerHTML` por `textContent`.



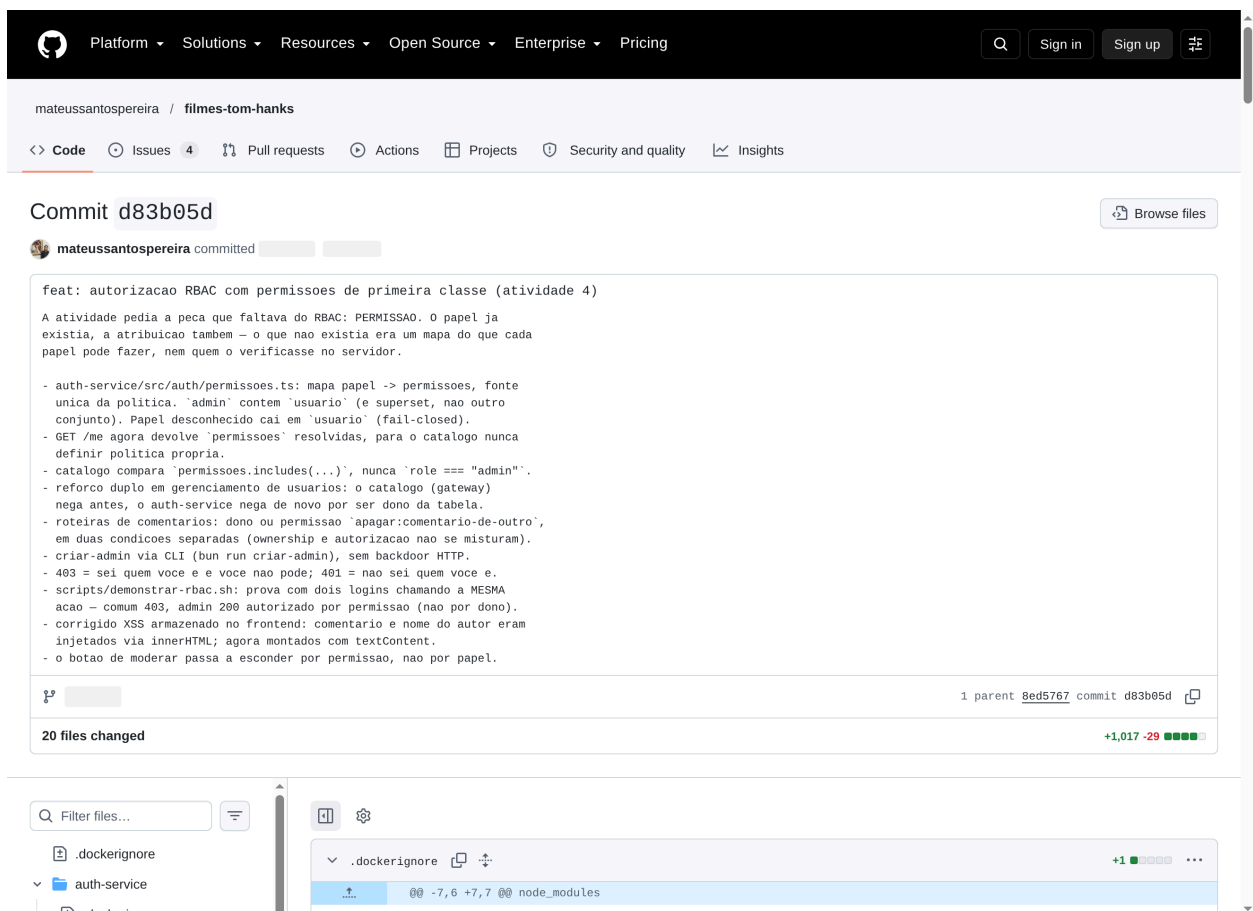
```
zsh - atv4 · data/hora do commit

mateus@fatec:~/filmes-tom-hanks$ git remote get-url origin
https://github.com/mateussantospereira/filmes-tom-hanks.git

mateus@fatec:~/filmes-tom-hanks$ git log -1 --date=format:"%d/%m/%Y %H:%M" --for
d83b05d | 06/10/2026 11:18

→ RBAC – controle de acesso por papel
```

Figura 10: Atividade 4 — evidência: commit d83b05d (terminal, 06/10/2026 11:18)



Platform Solutions Resources Open Source Enterprise Pricing

mateussantospereira / filmes-tom-hanks

<> Code Issues 4 Pull requests Actions Projects Security and quality Insights

Commit d83b05d

mateussantospereira committed

feat: autorizacao RBAC com permissoes de primeira classe (atividade 4)

A atividade pedia a peça que faltava do RBAC: PERMISSAO. O papel já existia, a atribuição também — o que não existia era um mapa do que cada papel pode fazer, nem quem o verificasse no servidor.

- auth-service/src/auth/permissoes.ts: mapa papel -> permissoes, fonte única da política. 'admin' contém 'usuario' (e superset, não outro conjunto). Papel desconhecido cai em 'usuario' (fail-closed).
- GET /me agora devolve 'permissoes' resolvidas, para o catálogo nunca definir política própria.
- catálogo compara 'permissoes.includes(...)', nunca 'role === "admin"'. reforço duplo em gerenciamento de usuários: o catálogo (gateway) nega antes, o auth-service nega de novo por ser dono da tabela.
- rotas de comentários: dono ou permissão 'apagar:comentario-de-outro', em duas condições separadas (ownership e autorizacao não se misturam).
- criar-admin via CLI (bun run criar-admin), sem backdoor HTTP.
- 403 = sei quem você é e você não pode; 401 = não sei quem você é.
- scripts/demonstrar-rbac.sh: prova com dois logins chamando a MESMA ação — comum 403, admin 200 autorizado por permissão (não por dono).
- corrigido XSS armazenado no frontend: comentário e nome do autor eram injetados via innerHTML; agora montados com textContent.
- o botão de moderar passa a esconder por permissão, não por papel.

1 parent 8ed5767 commit d83b05d

20 files changed +1,017 -29

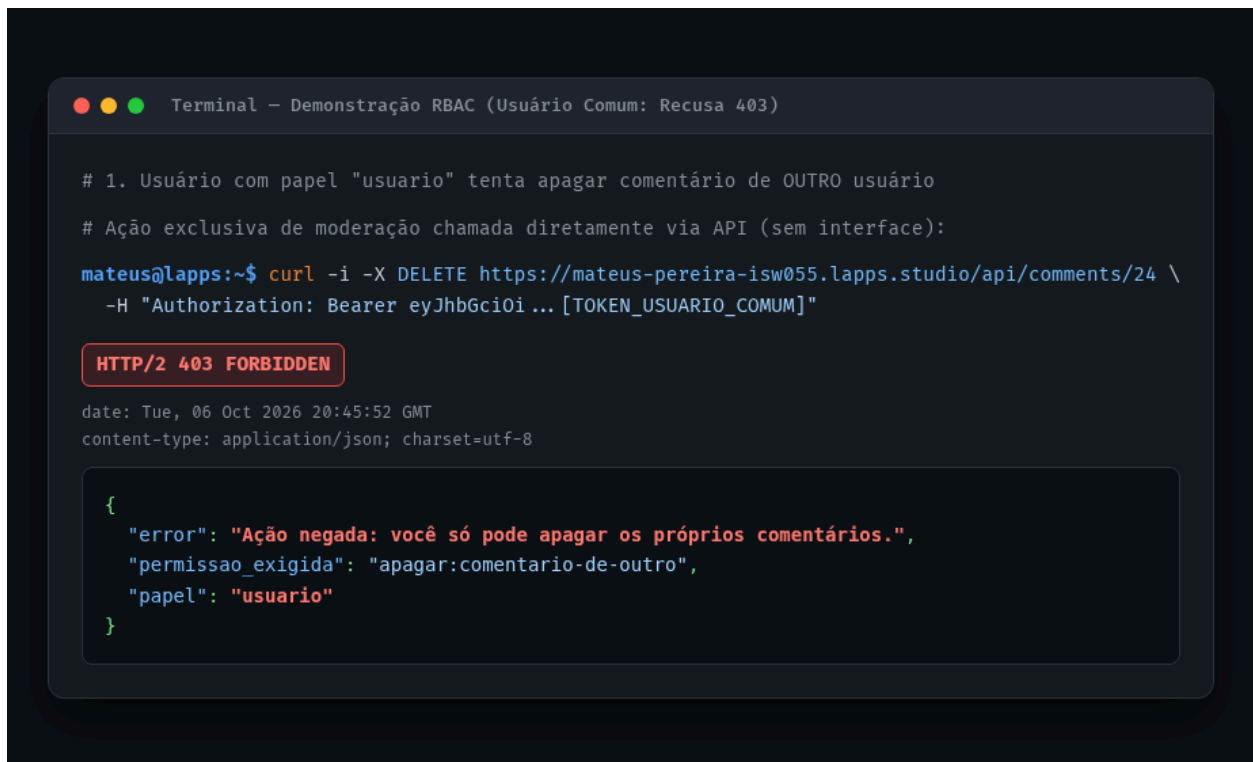
Filter files...

- .dockerignore
- auth-service
- .dockerignore

.dockerignore

node_modules

Figura 11: Atividade 4 — evidência: página do commit no GitHub



Terminal — Demonstração RBAC (Usuário Comum: Recusa 403)

```
# 1. Usuário com papel "usuario" tenta apagar comentário de OUTRO usuário
# Ação exclusiva de moderação chamada diretamente via API (sem interface):
mateus@lapps:~$ curl -i -X DELETE https://mateus-pereira-isw055.lapps.studio/api/comments/24 \
-H "Authorization: Bearer eyJhbGciOi ... [TOKEN_USUARIO_COMUM]"
```

HTTP/2 403 FORBIDDEN

date: Tue, 06 Oct 2026 20:45:52 GMT
content-type: application/json; charset=utf-8

```
{
  "error": "Ação negada: você só pode apagar os próprios comentários.",
  "permissao_exigida": "apagar:comentario-de-outro",
  "papel": "usuario"
}
```

Figura 12: Atividade 4 — resultado: 403 para usuário comum



Terminal — Demonstração RBAC (Administrador: Sucesso 200 por Permissão)

```
# 2. Administrador com papel "admin" tenta apagar o MESMO comentário de outro usuário
# O servidor autoriza a moderação pela permissão "apagar:comentario-de-outro":
mateus@lapps:~$ curl -i -X DELETE https://mateus-pereira-isw055.lapps.studio/api/comments/24 \
-H "Authorization: Bearer eyJhbGciOi ... [TOKEN_ADMIN]"
```

HTTP/2 200 OK

date: Tue, 06 Oct 2026 20:45:54 GMT
content-type: application/json; charset=utf-8

```
{
  "message": "Comentário removido",
  "autorizacao": "permissao:apagar:comentario-de-outro"
}
```

Figura 13: Atividade 4 — resultado: a mesma ação autorizada para admin

Dificuldades e como foram resolvidas. Essa atrasou porque fui desenvolvendo junto com as outras atividades. A parte mais chata foi provar a regra dos dois lados: a demonstração usa uma terceira conta como alvo, para a exclusão pelo admin acontecer por **permissão** e não por **ownership**. O XSS apareceu quando revisei o frontend e corrigi antes de entregar.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	06/10/2026 18:30
Situação	entregue com atraso
Evidência	GitHub — commit + print da consulta de logs pelo admin
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/dca3a82

O que foi feito. Criei um serviço de logs (log-service) que consome os eventos de um Redis Streams e deixa tudo pronto para consulta: login, ações sensíveis e tentativas negadas com 403. Um middleware global de auditoria pega qualquer `acao_negada` na borda do catálogo e publica no stream; o admin consulta os eventos pela tela de auditoria do sistema. Nas atividades seguintes o mesmo mecanismo passou a registrar também `upload_foto` e `editar_perfil`.

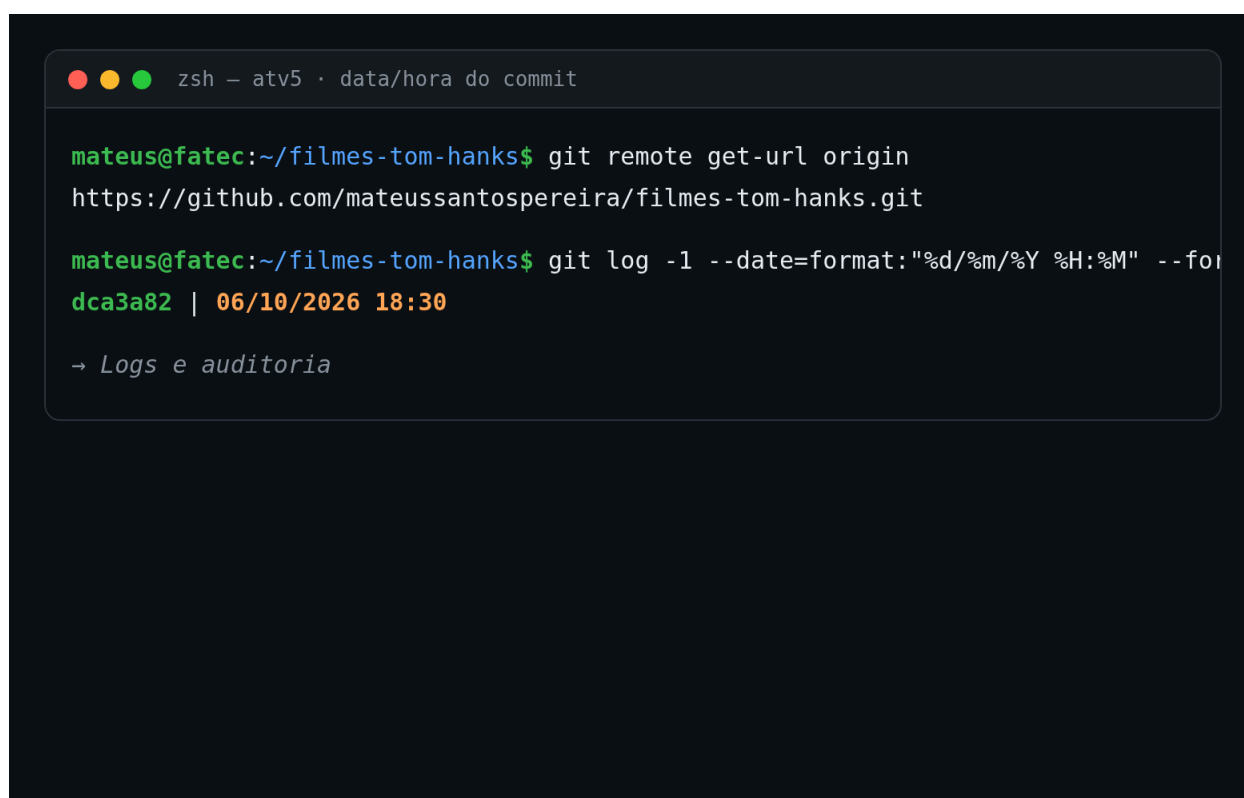
A terminal window with a dark background and light-colored text. The title bar shows 'zsh - atv5 · data/hora do commit'. The user 'mateus@fatec' is in the directory '~/filmes-tom-hanks'. They run 'git remote get-url origin' which returns 'https://github.com/mateussantospereira/filmes-tom-hanks.git'. Then they run 'git log -1 --date=format:"%d/%m/%Y %H:%M" --for dca3a82' which returns '06/10/2026 18:30'. Below the output, there is a link '→ Logs e auditoria'.

Figura 14: Atividade 5 — evidência: commit dca3a82 (terminal, 06/10/2026 18:30)

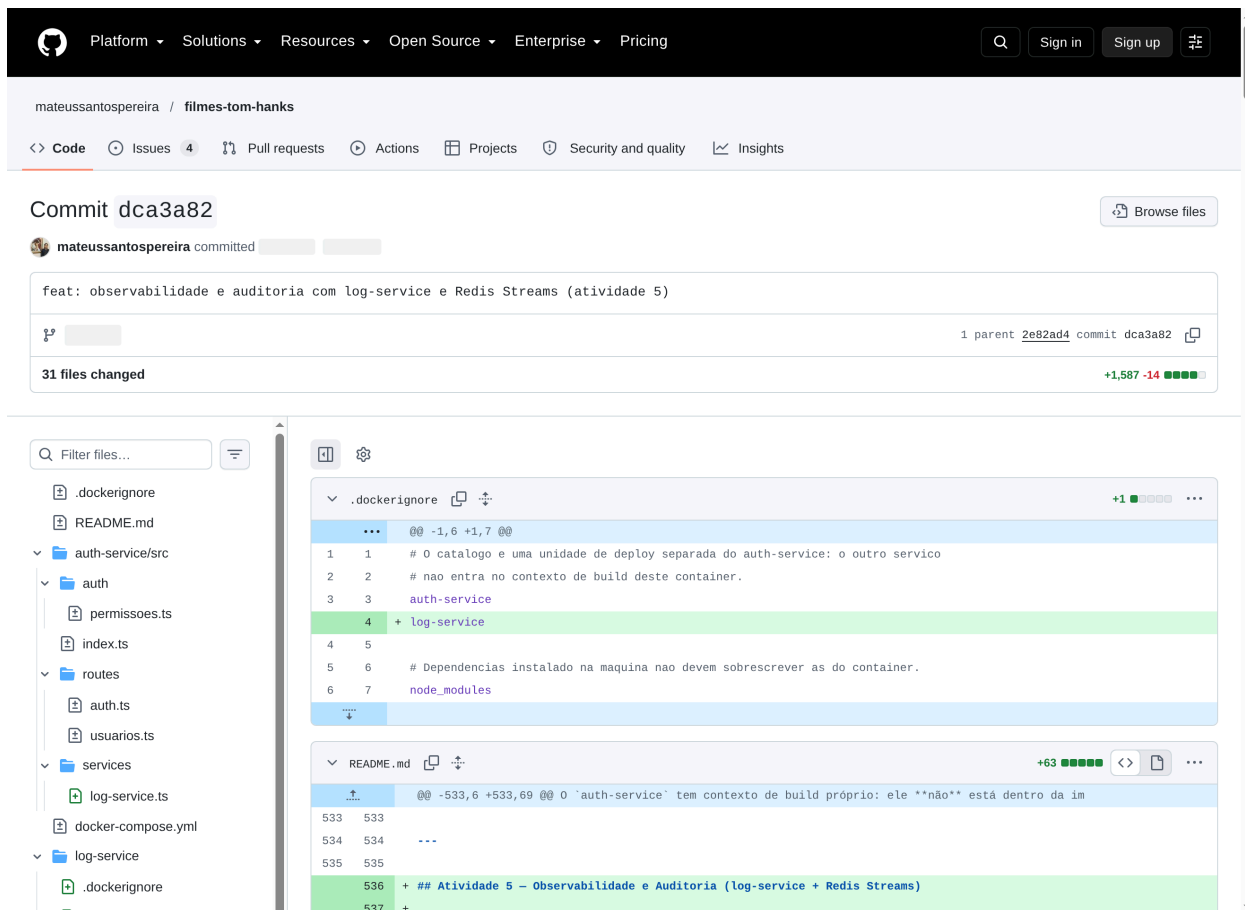


Figura 15: Atividade 5 — evidência: página do commit no GitHub

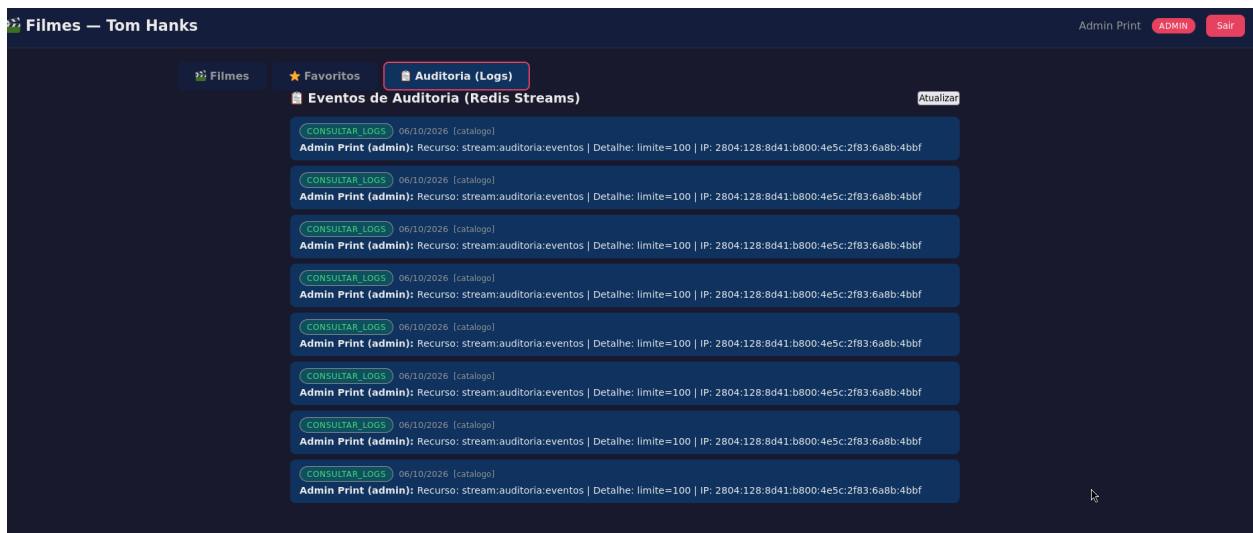


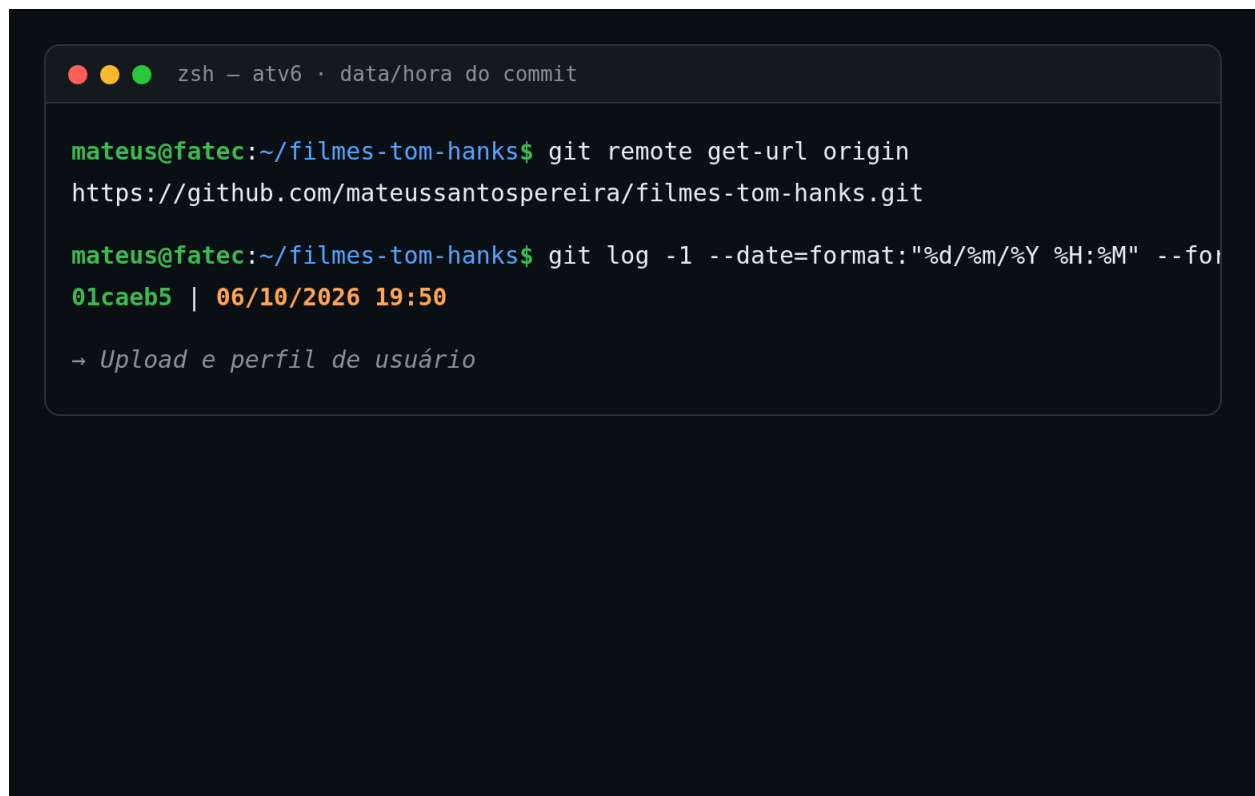
Figura 16: Atividade 5 — resultado: consulta dos logs de auditoria pelo admin

Dificuldades e como foram resolvidas. O Redis e o log-service não persistem volume, então o stream de auditoria zera a cada redeploy — comportamento esperado e documentado, mas a demonstração precisa rodar com os serviços ativos. Também tive de versionar os arquivos estáticos (?v=) para o cache do CDN não servir script velho.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	06/10/2026 19:50
Situação	entregue com atraso
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/01caeb5

O que foi feito. A página de perfil ganhou bio e foto: o arquivo enviado vai para o bucket privado perfis do MinIO (com volume no compose), a referência fica no MariaDB e a foto volta ao navegador por uma rota autenticada do catálogo (GET /api/perfil/:id/foto). O upload é validado no servidor: só imagem (415) e no máximo 2 MB (413). A identidade vem do token, nunca do ID que a pessoa manda no corpo ou na URL — editar ou fotografar o perfil de outra pessoa devolve 403 (permissao_exigida: "editar:perfil-proprio"), e o evento entra na auditoria como acao_negada.



```
zsh — atv6 · data/hora do commit

mateus@fatec:~/filmes-tom-hanks$ git remote get-url origin
https://github.com/mateussantospereira/filmes-tom-hanks.git

mateus@fatec:~/filmes-tom-hanks$ git log -1 --date=format:"%d/%m/%Y %H:%M" --for
01caeb5 | 06/10/2026 19:50

→ Upload e perfil de usuário
```

Figura 17: Atividade 6 — evidência: commit 01caeb5 (terminal, 06/10/2026 19:50)

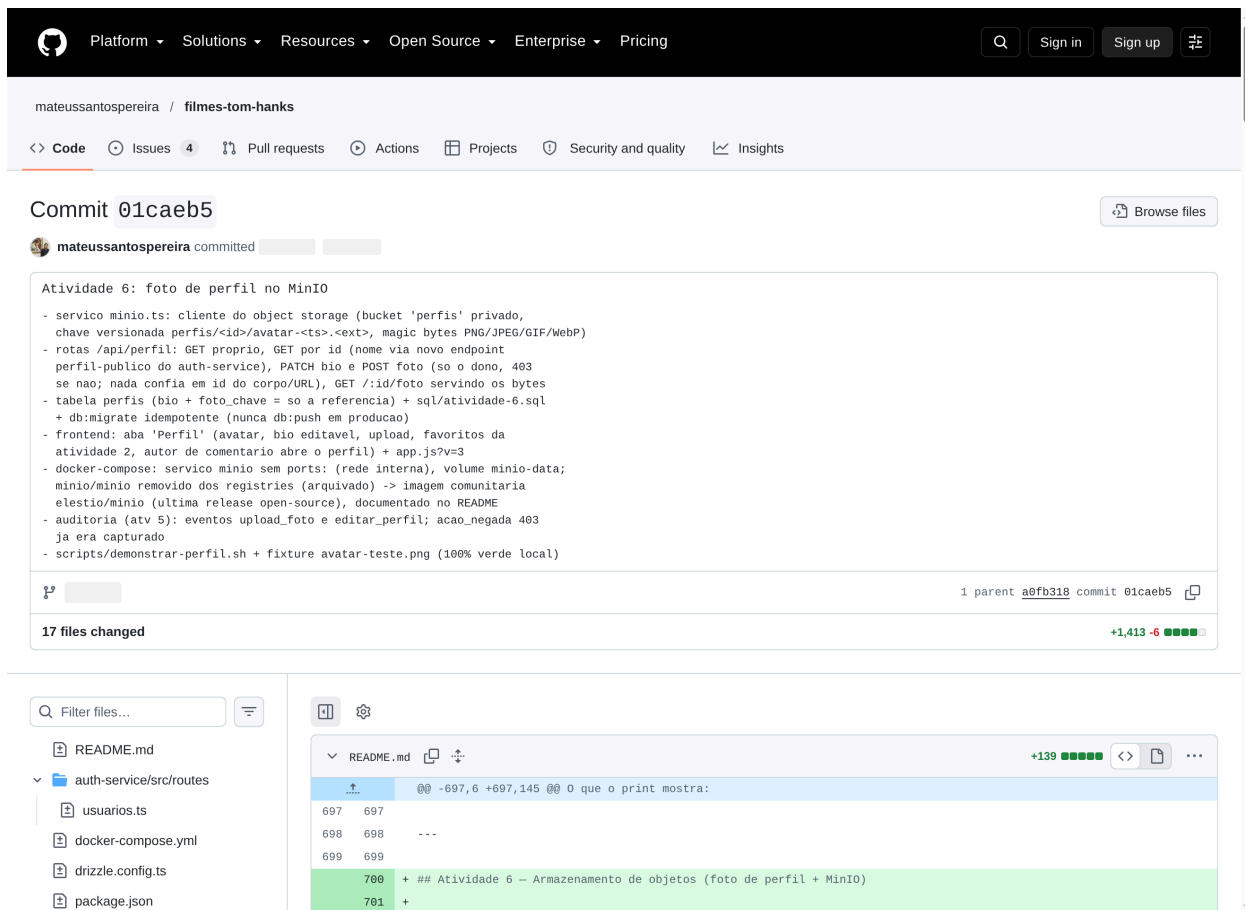


Figura 18: Atividade 6 — evidência: página do commit no GitHub

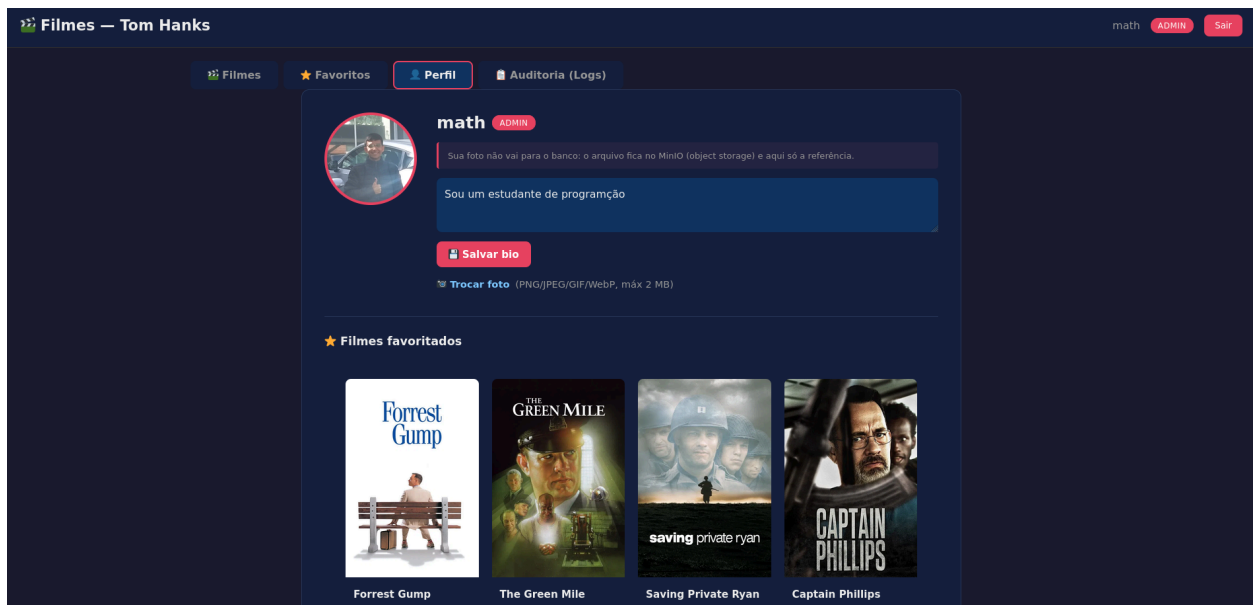


Figura 19: Atividade 6 — resultado: perfil com foto de upload no MinIO

```
bash - ./scripts/demonstrar-perfil.sh · saída real contra produção

mateus@lapps:~/filmes-tom-hanks$ CATALOGO_URL=https://mateus-pereira-isw055.lapps.studio ./scripts/demonstrar-perfil.sh

# A identidade do usuário NUNCA vem do corpo/URL: o servidor resolve o TOKEN pelo auth-service
# (GET /me) e compara com o id enviado — id de outra pessoa? 403 na borda, sem olhar o conteúdo.

PASSO 6 · TENTATIVA (RECUSADA) DE EDITAR O PERFIL DE OUTREM

✓ comum edita a BIO de outro (PATCH /api/perfil/52) 403

{
  "error": "Ação negada: você só pode editar o seu próprio perfil.",
  "permissao_exigida": "editar:perfil-proprio",
  "papel": "usuario",
  "id_recebido": 52,
  "sua_conta": 51
}

✓ comum envia FOTO para o perfil de outro (POST /api/perfil/52/foto) 403

{
  "error": "Ação negada: você só pode editar o seu próprio perfil.",
  "permissao_exigida": "editar:perfil-proprio",
  "papel": "usuario",
  "id_recebido": 52,
  "sua_conta": 51
}

✓ bio de outra pessoa segue intacta
✓ foto de outra pessoa segue ausente

PASSO 7 · AUDITORIA CAPTURA OS EVENTOS (CONTINUAÇÃO DA ATIVIDADE 5)

# Upload e edição geram eventos novos no mesmo stream do Redis; os 403 viram acao_negada.
# Conferido no log de auditoria (log-service + emissão via /api/logs).

✓ evento 'upload_foto' presente no log de auditoria
✓ evento 'editar_perfil' presente no log de auditoria
✓ evento 'acao_negada' presente no log de auditoria

TODAS AS CONFERENCIAS DA ATIVIDADE 6 PASSARAM (exit 0)
```

Figura 20: Atividade 6 — resultado: recusa de editar perfil alheio (403)

Dificuldades e como foram resolvidas. No primeiro deploy a foto não aparecia: `background-image` no CSS não manda o header `Authorization`, então a rota autenticada devolvia 401. Resolvi carregando a foto com `fetch` mandando o token e montando uma blob URL (`URL.createObjectURL`) para o avatar. Depois, uma entrada velha do cache do CDN fez o `?v=4` não atualizar, e subi para `?v=5` — aprendi a nunca pré-aquecer a URL nova antes do redeploy terminar.

5 Considerações finais

Durante o bimestre eu percebi na prática a diferença entre “funcionar na minha máquina” e “funcionar em produção”. A parte mais difícil foi operar o sistema em contêineres: rede interna entre serviços, variáveis

de ambiente diferentes, cache de CDN servindo versão velha e serviço sem volume persistente zerando os logs. Cada problema virou uma lição que deixei anotada no README, e hoje eu já antecipo essas coisas antes de publicar.

O ritmo não foi uniforme: o catálogo saiu até antes do prazo, mas desacoplar o login e principalmente RBAC e auditoria atrasaram e fecharam na reta final. Se eu fosse refazer, dividiria cada atividade em partes menores e faria o deploy logo depois de cada uma, em vez de concentrar o fechamento no fim do período. Mesmo assim, as seis atividades estão entregues e comprovadas, com scripts de demonstração rodando contra o ambiente real.

Para o próximo bimestre levo a rotina de validar com scripts automáticos (saída verificável) e o hábito de anotar as dificuldades na hora — foi isso que deixou este relatório com datas e evidências reais, sem depender da memória.

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Mateus dos Santos Pereira

Pompeia, 07/10/2026